Lecture 05: Assignment 4 Problem Set & Systems Case Studies

Welcome to the official problem specification catalog for **Lecture 05: Object-Oriented Programming (OOP) in Python**. This document contains complete specifications, mathematical formulations, structural design requirements, and test vectors for all 10 Assignment 4 problems and 3 systems engineering case studies.

📋 Table of Contents

- Problem 01: BankAccount Encapsulated Ledger Class
- Problem 02: Book Entity with Review Aggregation
- Problem 03: Encapsulated Student Record with Validated Accessors
- Problem 04: Polymorphic Shape Hierarchy & Area Dynamic Dispatch
- Problem 05: Vehicle Taxonomy Inheritance & Attribute Extension
- Problem 06: Abstract Employee Payroll Contract
- Problem 07: Constructor Overloading Simulation via Optional Defaults
- Problem 08: Global Game Player Population Tracker
- Problem 09: Multiple Inheritance & Omnivore Dietary Synthesis
- Problem 10: Mini-Project – Multi-Entity OOP Chat System
- Systems Case Study 1: CPython Object Memory & `__slots__` Architecture
- Systems Case Study 2: Diamond Inheritance & C3 Linearization (MRO)
- Systems Case Study 3: Dynamic Attribute Lookup Protocol & Descriptors

---

Problem 01: BankAccount Encapsulated Ledger Class

Problem Description

Create a class `BankAccount` with attributes `account_number: str`, `owner_name: str`, and `balance: float`. Implement the following methods:

1. `deposit(amount: float) -> bool`: Increases balance by `amount` if `amount > 0`.
2. `withdraw(amount: float) -> bool`: Decreases balance by `amount` if funds are sufficient ($amount \leq balance$) and $amount > 0$; otherwise rejects with a warning message.
3. `check_balance() -> float`: Returns the current available balance.

Mathematical Invariants

Let B_t denote the balance at transaction step t:

$$B_{t+1} = \begin{cases} B_t + \Delta & \text{if deposit of } \Delta > 0 \\ B_t - \Delta & \text{if withdrawal of } 0 < \Delta \leq B_t \\ B_t & \text{otherwise (invalid transaction)} \end{cases}$$

Strict non-negativity constraint:

$$\forall t \geq 0, \quad B_t \geq 0$$

Problem 02: Book Entity with Review Aggregation

Problem Description

Create a class `Book` with attributes `title: str`, `author: str`, and an internal collection `reviews: list[str]`. Implement methods:

1. `add_review(review: str) -> None`: Appends a new review string to the collection.
2. `count_reviews() -> int`: Returns the total number of reviews recorded.
3. `display_reviews() -> None`: Prints all reviews in formatted chronological order.

Mathematical Formulation

Let the reviews sequence be $R = (r_1, r_2, \ldots, r_K)$ where $r_i \in \Sigma^*$. The cardinality metric is:

$$|R| = \sum_{i=1}^{K} 1 = K$$

Problem 03: Encapsulated Student Record with Validated Accessors

Problem Description

Create a class `Student` with private attributes `_name`, `_roll_no`, and `_marks`. Provide Pythonic getter and setter properties (`@property` and `@setter`) enforcing the following structural validation invariants:

- `name`: Must be a non-empty string after stripping whitespace ($|\text{name}| > 0$).
- `roll_no`: Must be an integer in the closed interval $[1, 100]$.
- `marks`: Must be non-negative ($\text{marks} \geq 0.0$).

Violations must raise appropriate `ValueError` exceptions with descriptive error messages.

Problem 04: Polymorphic Shape Hierarchy & Area Dynamic Dispatch

Problem Description

Create a base class `Shape` defining an abstract or interface method `area() -> float`. Create three distinct subclasses that override the `area()` method:

1. `Circle(radius: float)`: Computes $\mathcal{A}_{\text{circle}} = \pi r^2$.
2. `Rectangle(width: float, height: float)`: Computes $\mathcal{A}_{\text{rect}} = w \cdot h$.
3. `Triangle(base: float, height: float)`: Computes $\mathcal{A}_{\text{tri}} = \frac{1}{2} b \cdot h$.

Demonstrate polymorphic execution by iterating over a heterogeneous collection of shapes and invoking `.area()` uniformly.

Mathematical Formulation

$$\mathcal{A}(\text{Shape}) = \begin{cases} \pi r^2 & \text{if Shape is Circle} \\ w \cdot h & \text{if Shape is Rectangle} \\ \frac{1}{2} b \cdot h & \text{if Shape is Triangle} \end{cases}$$

Problem 05: Vehicle Taxonomy Inheritance & Attribute Extension

Problem Description

Create a generalized base class `Vehicle` with common attributes `brand: str` and `model: str`. Derive two specialized subclasses:

1. `Car(Vehicle)`: Extends the base constructor via `super()` to accept and store an additional attribute `seats: int`.

2. `Bike(Vehicle)`: Extends the base constructor via `super()` to accept and store an additional attribute `engine_cc: int`.

Implement a `describe()` method in each class illustrating hierarchical method overriding and `super()` delegation.

Problem 06: Abstract Employee Payroll Contract

Problem Description

Using Python's `abc` module (`ABC`, `@abstractmethod`), define an abstract base class `Employee` with an abstract method `calculate_salary() -> float`. Derive three concrete subclasses with distinct salary computation rules:

1. `Intern(Employee)`: Receives a fixed monthly stipend.
2. `FullTimeEmployee(Employee)`: Receives a base monthly salary plus performance bonus ($S = \text{base} + \text{bonus}$).
3. `ContractEmployee(Employee)`: Receives compensation proportional to hours worked ($S = \text{hourly_rate} \times \text{hours}$).

Verify that attempting to instantiate `Employee` directly raises a `TypeError`.

Problem 07: Constructor Overloading Simulation via Optional Defaults

Problem Description

Because Python does not support multiple `__init__` declarations, create a class `Person` that simulates constructor overloading using default parameter arguments. The constructor must seamlessly accommodate all three usage patterns:

1. Instantiation with `name` only: `Person("Alice")`
2. Instantiation with `name` and `age`: `Person("Bob", 25)`
3. Instantiation with `name`, `age`, and `address`: `Person("Charlie", 30, "742 Evergreen Terrace")`

Problem 08: Global Game Player Population Tracker

Problem Description

Create a class `Player` with:

- A class-level attribute `player_count: int` initialized to 0.
- Instance-level attributes `name: str` and `level: int`.

Each time a new `Player` instance is constructed, automatically increment `Player.player_count` by 1. Provide a `@classmethod` to retrieve the active player population count.

Problem 09: Multiple Inheritance & Omnivore Dietary Synthesis

Problem Description

Demonstrate multiple inheritance and Method Resolution Order (MRO) by implementing:

1. Base class `Herbivore` with attribute `diet = "plants"` and method `forage()`.
2. Base class `Carnivore` with attribute `diet = "meat"` and method `hunt()`.
3. Subclass `Bear` (or `Omnivore`) inheriting simultaneously from both `Herbivore` and `Carnivore`, synthesizing both behaviors.

Inspect and document `Bear.mro()` to explain how Python resolves method conflicts.

Problem 10: Mini-Project – Multi-Entity OOP Chat System

Problem Description

Design and implement a complete Object-Oriented Chat System consisting of three interconnected domain entities:

1. `Message`: Stores an auto-incrementing `id: int`, `sender: User`, `content: str`, and timestamp. Implements `__str__()` returning `(id) username: content`.
2. `User`: Stores `username: str` and current `chatroom: Optional[ChatRoom]`. Implements `join_chatroom(room)`, `leave_chatroom()`, and `send_message(content)`.
3. `ChatRoom`: Stores `name: str`, `users: list[User]`, and `messages: list[Message]`. Implements `add_user()`, `remove_user()`, `broadcast(sender, content)`, and `show_chat_history()`.

Systems Case Study 1: CPython Object Memory & __slots__ Architecture

Problem Statement

Investigate CPython's internal instance memory allocation. Profile the memory footprint of standard instances (allocating dynamic `__dict__` hash tables) versus optimized `__slots__` instances (allocating compact fixed-size C pointer arrays). Quantify heap memory savings across $N = 10^5$ objects.

Systems Case Study 2: Diamond Inheritance & C3 Linearization (MRO)

Problem Statement

Analyze the C3 Linearization algorithm used by Python to compute Method Resolution Order (MRO). Prove why C3 guarantees local precedence order and monotonicity, preventing the classic Diamond Problem ambiguities found in early object-oriented languages.

Systems Case Study 3: Dynamic Attribute Lookup Protocol & Descriptors

Problem Statement

Dissect CPython's complete attribute lookup hierarchy (`__getattribute__`). Trace the priority order across Data Descriptors, Instance `__dict__` , Non-Data Descriptors (Methods), Class `__dict__` , and the fallback hook `__getattr__` .